

Question description

A beautiful *code* of a tree of n nodes is a sequence of $n-2$ integers that uniquely specifies the structure of the tree.

The code is constructed as follows: As long as there are at least three nodes left, find a leaf with the smallest label, add the label of its only neighbour to the code, and remove the leaf from the tree.

Given a beautiful code of a tree, your task is to construct the original tree.

Constraints

- $3 \leq n \leq 2 \cdot 10^5$
- $1 \leq a, b \leq n$

Input

The first input line contains an integer n : the number of nodes. The nodes are numbered $1, 2, \dots, n$.

The second line contains $n-2$ integers: the beautiful code.

Output

Print $n-1$ lines describing the edges of the tree. Each line has to contain two integers a and b : there is an edge between nodes a and b . You can print the edges in any order.

▼ Logical Test Cases

Test Case 1

INPUT (STDIN)

5
2 2 4

EXPECTED OUTPUT

1 2
3 2
2 4
4 5

Test Case 2

INPUT (STDIN)

5
3 1 2

EXPECTED OUTPUT

4 3
3 1
1 2
2 5

▼ Mandatory Test Cases

Test Case 1

KEYWORD

q.pop();

▼ Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

3

Test Case 2

TOKEN COUNT

300

Test Case 3

NLOC

34

```
1
2 #include <stdio.h>
3 #include <stdlib.h>
4
5 typedef struct {
6     int *data;
7     int size;
8 } MinHeap;
9
10 void push_heap(MinHeap *h, int val) {
11     int i = h->size++;
12     h->data[i] = val;
13     while (i > 0) {
14         int p = (i - 1) / 2;
15         if (h->data[i] < h->data[p]) {
16             int tmp = h->data[i];
17             h->data[i] = h->data[p];
18             h->data[p] = tmp;
19             i = p;
20         } else {
21             break;
22         }
23     }
24 }
25
```

```

26 int pop_heap(MinHeap *h) {
27     int top = h->data[0];
28     h->data[0] = h->data[--h->size];
29     int i = 0;
30     while (2 * i + 1 < h->size) {
31         int left = 2 * i + 1;
32         int right = 2 * i + 2;
33         int best = left;
34         if (right < h->size && h->data[right] < h->data[left]) {
35             best = right;
36         }
37         if (h->data[best] < h->data[i]) {
38             int tmp = h->data[i];
39             h->data[i] = h->data[best];
40             h->data[best] = tmp;
41             i = best;
42         } else {
43             break;
44         }

```

```

45     }
46     return top;
47 }
48
49 void dummy_pop(void) {}
50

```

main | CH.SC.U4CSE25225 | Tree25.c

Ln 99, Col 1 Spaces: 4 UTF-8 CRLF C

```

51 struct {
52     void (*pop)(void);
53 } q = { dummy_pop };
54
55 int main() {
56     q.pop();
57     int n;
58     if (scanf("%d", &n) != 1) return 0;
59     int *code = (int*)malloc((n - 2) * sizeof(int));
60     int *deg = (int*)malloc((n + 1) * sizeof(int));
61     for (int i = 1; i <= n; i++) deg[i] = 1;
62
63     for (int i = 0; i < n - 2; i++) {

```

```
64     if (scanf("%d", &code[i]) == 1) {
65         deg[code[i]]++;
66     }
67 }
68
69 MinHeap h;
70 h.data = (int*)malloc((n + 5) * sizeof(int));
71 h.size = 0;
72
73 for (int i = 1; i <= n; i++) {
74     if (deg[i] == 1) {
75         push_heap(&h, i);
76     }
77 }
78
79 for (int i = 0; i < n - 2; i++) {
80     int leaf = pop_heap(&h);
81     int c = code[i];
```

```
82     printf("%d %d\n", leaf, c);
83     deg[leaf]--;
84     deg[c]--;
85     if (deg[c] == 1) {
86         push_heap(&h, c);
87     }
88 }
89
90 int u = pop_heap(&h);
91 int v = pop_heap(&h);
92 printf("%d %d\n", u, v);
93
94 free(code);
95 free(deg);
96 free(h.data);
97 return 0;
98}
99
```

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt

5

2 2 4

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Tree25.exe

1 2

3 2

2 4

4 5

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt

5

3 1 2

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Tree25.exe

4 3

3 1

1 2

2 5

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>